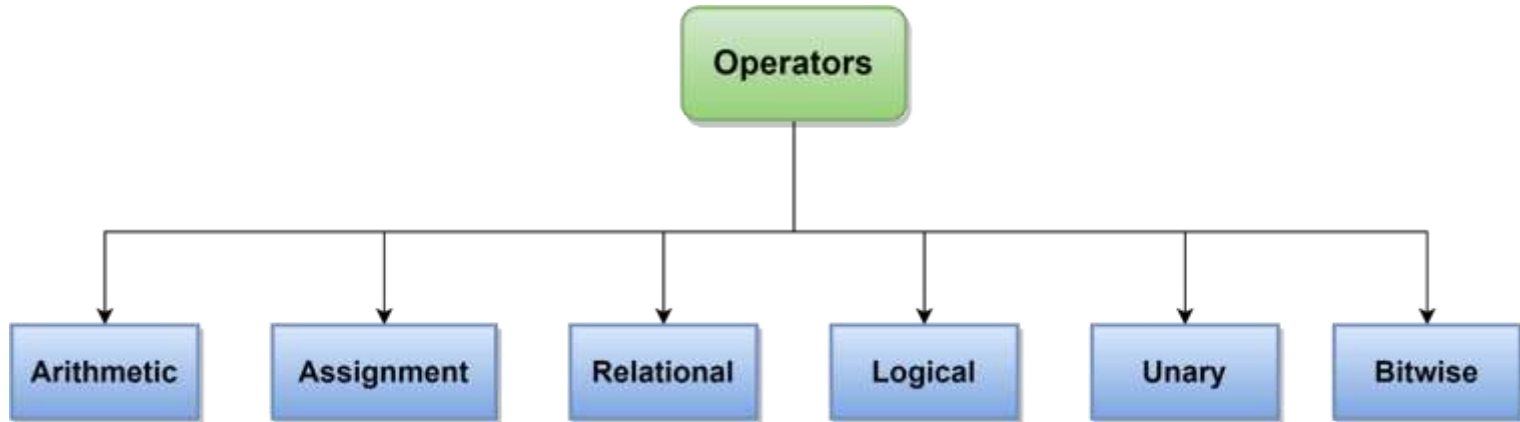


Introduction to Operators

Operators

- Operators are symbols used to perform specific operations on one or more operands to produce a result.
- Types of operators in Java:



Arithmetic Operators

Category	Name	Symbol
Additive	Addition	+
	Subtraction	-
Multiplicative	Multiplication	*
	Division	/
	Modulus	%

Arithmetic Operators (Continued)

- **Addition:** Adds two variables/numbers of same datatype

- Example:

```
System.out.println(2+3); //Output: 5
```

- **Subtraction:** Subtracts the second number from the first one

- Example:

```
System.out.println(7-3); //Output: 4
```

- **Multiplication:** Multiplies the first operand with the second one

- Example:

```
System.out.println(3*3); //Output: 9
```

Arithmetic Operators (Continued)

- **Division:** Divides the first operand with the second one
 - Example:

```
System.out.println(9/2);    //Output: 4  
System.out.println(9.0/2.0); //Output: 4.5
```

- **Modulus:** Finds the remainder after dividing the first number with the second
 - Example:

```
System.out.println(5%2);    //Output: 1  
System.out.println(7%9);    //Output: 7
```

Assignment Operators

Name	Symbol	Identity
Assignment	=	a = 5;
Addition Equals	+=	a += 3; a = a+3;
Subtraction Equals	-=	a -= 3; a = a-3;
Multiplication Equals	*=	a *= 2; a = a*2;
Division Equals	/=	a /= 2; a = a/2;
Modulus Equals	%=	a %= 2; a = a%2;

Assignment Operators (Continued)

- **(=)**: Assigns the value on the right to the variable on the left.

- Example:

```
int number = 10;  
float value = 2.5f;
```

- **(+=)**: Adds both the operands on its right and left and assigns the addition to the operand on its left .

- Example:

```
int a = 5, b = 10;  
b += a;  
System.out.println(b); // Output: 15
```

Assignment Operators (Continued)

- **(-=)**: Subtracts the right operand from the left operand and assigns the result to the left operand.

- Example:

```
int a = 5, b = 10;  
b -= a;  
System.out.println(b); // Output: 5
```

- **(*=)**: Multiplies both the operands and assigns the result to the left operand.

- Example:

```
int a = 5, b = 10;  
b *= a;  
System.out.println(b); // Output: 50
```

Assignment Operators (Continued)

- **(/=)**: Divides the operand on its left with the operand on its right and assigns the result to the left operand

- Example:

```
int a = 5, b = 10;  
b /= a;  
System.out.println(b); // Output: 2
```

- **(%=)**: Divides the left operand by the right operand and assigns the remainder to the operand on the left.

- Example:

```
int a = 5, b = 10;  
b %= a;  
System.out.println(b); // Output: 0
```

Relational Operators

Category	Name	Symbol
Comparison	Greater Than	>
	Less Than	<
	Greater Than or Equals	>=
	Less Than or Equals	<=
Equality	Equals	==
	Not Equals	!=

Relational Operators (Continued)

- (**>**): Checks if the left operand is greater than the right operand.
 - Example:

```
System.out.println(5>3); // Output: true
```

- (**<**): Checks if the left operand is less than the right operand.
 - Example:

```
System.out.println(5<3); // Output: false
```

- (**>=**): Checks if the left operand is greater than or equals to the right operand.
 - Example:

```
System.out.println(7>=10); // Output: false
```

Relational Operators (Continued)

- (**<=**): Checks if the left operand is less than or equals to the right operand.
 - Example:

```
System.out.println(2<=3); // Output: true
```

- (**==**): Checks if both the operands are equal.
 - Example:

```
System.out.println(5==3); // Output: false
```

- (**!=**): Checks if the left operand is not equals to the right operand.
 - Example:

```
System.out.println(7!=10); // Output: true
```

Logical Operators

Name	Symbol
Logical AND	&&
Logical OR	
Logical NOT	!

Logical Operators (Continued)

- **(&&):** Denotes true if both the conditions before and after are true, otherwise false.
 - Example: false && true

```
int a = 5, b = 10, c = 20;  
System.out.println(a>b && b<c); // Output: false
```

- **(||):** Denotes true if at least one conditions before and after are true, otherwise false.
 - Example: false || true

```
System.out.println(5<-1 || 10>5); // Output: true
```

- **(!):** Reverses the logical value of the operand on its right.
 - Example: !(true)

```
System.out.println(!(5>3)); // Output: false
```

Unary Operators

Type	Name	Symbol
Prefix	Negation	-
	Not	!
	Increment	++
	Decrement	--
Postfix	Increment	++
	Decrement	--

Unary Operators (Continued)

- **Negation (-):** Negates an operand
 - Example:

```
int a = 5;  
int b = -a;  
System.out.println(b); // Output: -5
```

- **Not (!):** Reverses the logical value of an operand
 - Example:

```
System.out.println(! false); // Output: true
```

Unary Operators (Continued)

- **Pre-Increment (++):** Increments the operand by 1 and then assigned.
 - Example:

```
int a = 5;  
int y = ++a; // a+=1; y=a;  
System.out.println(a); // Output: 6  
System.out.println(y); // Output: 6
```

- **Pre-Decrement (--):** Decrements the operand by 1 and then assigned.
 - Example:

```
int a = 5;  
int y = --a; // a-=1; y=a;  
System.out.println(a); // Output: 4  
System.out.println(y); // Output: 4
```

Unary Operators (Continued)

- **Post-Increment (++)**: The value is assigned first and then increments the operand by 1.
 - Example:

```
int a = 5;  
int y = a++; // y=a; a+=1;  
System.out.println(a); // Output: 6  
System.out.println(y); // Output: 5
```

- **Post-Decrement (--)**: The value is assigned first and then decrements the operand by 1.
 - Example:

```
int a = 5;  
int y = a--; // y=a; a-=1;  
System.out.println(y); // Output: 5  
System.out.println(a); // Output: 4
```

4.8 OPERATOR PRECEDENCE

The precedence is higher at the top and decreases as we move towards the bottom. If multiple operators with the same precedence are placed adjacently, operators are executed from left to right. If the first brackets “()” are present, we must go to the innermost first bracket and gradually move towards the outer ones.

Operator Type	Operator Category/Name	Symbol
Unary	Postfix	++, --
	Prefix	++, --, !, -
Arithmetic	Multiplicative	*, /, %
	Additive	+, -
Relational	Comparison	<, >, <=, >=
	Equality	==, !=
Bitwise	Bitwise AND	&
	Bitwise Exclusive OR	^
	Bitwise Inclusive OR	
Logical	Logical AND	&&
	Logical OR	
Assignment	All assignment operators	=, +=, -=, *=, /=, %=

Now, let us find the output of the following Java code lines:

Java Code Block	Output
<code>System.out.println(5*(5+5/5%(5*5)));</code>	30 Here 5*5 was executed first, then 5/5.
<code>System.out.println(5+1==(5+5/5%(5*5)));</code>	true
<code>System.out.println(-(-5-1)==(5+5/5%(5*5)));</code>	true Here -5-1 and 5*5 were executed first.
<code>System.out.println(5!=5 && 5%1!=1 1==(5/2));</code>	false

From the above lines of codes, we can see that if operators with same precedence are placed adjacent such as 5/5*5, we must calculate from left to right. Which means performing the division first and then the multiplication.

THANK YOU!